



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of
Computing

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 2 (10%) – Transaction Management

Name	:	CHUA JIA LIN , JOANNE CHING YIN XUAN
Matric Id	:	A23CS0069 , A23CS0227
Section	:	1

Instructions:

This assignment evaluates your ability to apply the concept of serializability, differentiate between serial and non-serial transaction schedules and implement locking techniques to manage simultaneous transactions.

Academic Integrity

All students must work in a group of **TWO(2)**. The use of AI-generated tools or sharing of answers between students **is strictly prohibited** and will result in **zero marks**. You are required to complete the answers in **handwritten form**. Please take **clear screenshots** of your answers and submit them in **PDF format**.

QUESTION 1 (15 MARKS)

Consider the schedule transactions in Table 1 and answer all following questions (i) – (iv).

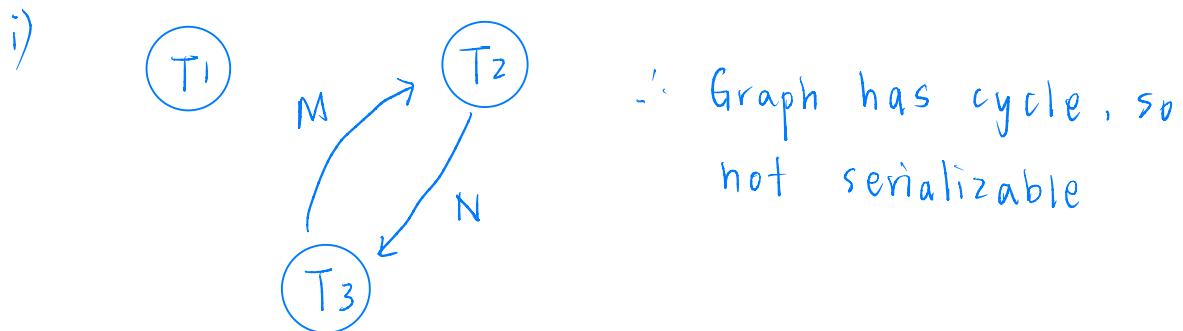
Table 1: Schedule T1, T2 and T3

Time	T1	T2	T3
t ₁	begin transaction		
t ₂	read_item(X)		
t ₃	read_item(Y)	begin transaction	
t ₄	commit	read_item(M)	begin transaction
t ₅		read_item(N)	read_item(N)
t ₆		$M = (N+M) * 10$	read_item(M)
t ₇		write_item(M)	$N = N + 150$
t ₈		commit	write_item(N)
t ₉			commit

- Draw the precedence graph for the transaction schedule in Table 1 to check for serializability.
- Explain the precedence graph produced in (i).
- Apply the locking technique to the transaction schedule in Table 1, and revise the transaction schedule in Table 1 using the following format:

T1	T2	T3

- Identify the problem(s) that arise in (iii) and then explain any potential problems that may still occur after applying the locking technique in concurrency control based on your answer in (iii).



- The precedence graph contains edges $T2 \rightarrow T3$ and $T3 \rightarrow T2$, forming a cycle. This indicates a circular dependency between T2 and T3. Therefore, schedule is not serializable. T1 is independent.

Table 1: Schedule T1, T2 and T3

Time	T1	T2	T3
t ₁	begin transaction		
t ₂	read_item(X)		
t ₃	read_item(Y)	begin transaction	
t ₄	commit	read_item(M)	begin transaction
t ₅		read_item(N) S-lock(N)	read_item(N) S
t ₆		M=(N+M)*10	read_item(M) S
t ₇		write_item(M) X-lock(M)	N=N+150
t ₈		commit	write_item(N) X
t ₉			commit

iii)

Time	T1	T2	T3
t ₁	begin transaction		
t ₂	S-lock(X)		
t ₃	read-item(X)	begin transaction	
t ₄	unlock(X)	X-lock(M)	begin transaction
t ₅	S-lock(Y)	read-item(M)	request X-lock(N)
t ₆	read-item(Y)	S-lock(N)	WAIT
t ₇	commit, unlock(Y)	read-item(N)	WAIT
t ₈		unlock(N)	WAIT
t ₉		M = (N + M) * 10	X-lock(N)
t ₁₀		write-item(M)	read-item(N)
t ₁₁		commit, unlock(M)	S-lock(M)
t ₁₂			WAIT
t ₁₃			read-item(M)
t ₁₄			unlock(M)
t ₁₅			N = N + 150
t ₁₆			write-item(M)
t ₁₇			commit, unlock(N)

iv) The problem arise in (iii) is waiting which is delay due to locking. T₃ has to wait T₂ because T₂ hold shared lock on N. The potential problem that will still occur is deadlock if 2 transactions hold locks that other needs. This cause both transaction unable to proceed. It will also cause starvation where transactions may repeatedly delayed when shared locks are continuously granted other transaction, a transaction that requesting an exclusive lock may wait indefinitely.

QUESTION 2 (15 MARKS)

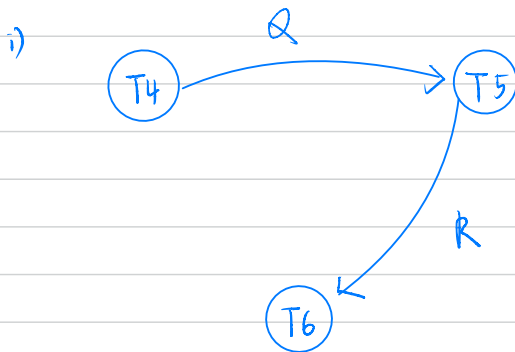
Based on the transaction schedule in Table 2, answer all the following questions:

Table 2: Transaction Schedule for T4-T5-T6

	T4	T5	T6
↓ time	begin transaction		
	read_item(P)		
	read_item(Q)	begin transaction	
	$P = (P * 5) + Q$	read_item(Q)	
	write_item(P)	read_item(R)	begin transaction
	commit	$Q = Q + R$	read_item(R)
		write_item(Q)	$R = R - 100$
		commit	write_item(R)
			commit

- Draw the precedence graph for the transaction schedule in Table 2 to check for serializability.
- Explain the precedence graph produced in (i).
- Apply the two-phase locking (2PL) protocol to emphasize the positioning of lock and unlock operations in each transaction using the following format.

T4	T5	T6



∴ No cycle, schedule is conflict - serializable

- ii) T4 reads R, and later T5 updates (write R).
 T5 reads R, and later T6 updates (write R).
 There is no cycle, the schedule is serial order.

Table 2: Transaction Schedule for T4-T5-T6

↓ time	T4	T5	T6
	begin transaction		
	read_item(P)		
	read_item(Q)	begin transaction	
	$P = (P * 5) + Q$	read_item(Q)	
	write_item(P)	read_item(R)	begin transaction
	commit	$Q = Q + R$	read_item(R)
		write_item(Q)	$R = R - 100$
		commit	write_item(R)
			commit

(iii)

Time	T4	T5	T6
t_1	begin transaction		
t_2	write-lock(P)		
t_3	read-item(P)	begin transaction	
t_4	read-lock(Q)	write-lock(Q)	
t_5	read-item(Q)	WAIT	begin transaction
t_6	$P = (P * 5) + Q$	WAIT	write-item(Q)
t_7	write-item(P)	WAIT	WAIT
t_8	commit, unlock(P, Q)	WAIT	WAIT
t_9		read-item(Q)	WAIT
t_{10}		read-lock(R)	WAIT
t_{11}		read-item(R)	WAIT
t_{12}		$Q = Q + R$	WAIT
t_{13}		write-item(Q)	WAIT
t_{14}		commit, unlock(Q, R)	WAIT
t_{15}			read-item(R)
t_{16}			$R = R - 100$
t_{17}			write-item(R)
t_{18}			commit-unlock(R)

QUESTION 3 (25 MARKS)

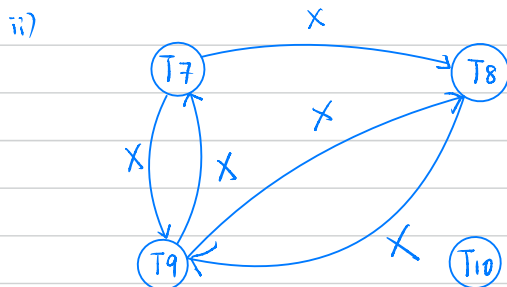
- a) Consider the schedule transactions in Table 3.1 and answer all following questions (i) – (iv).

Table 3.1: Schedule T7, T8, T9 and T10

Time	T7	T8	T9	T10
t ₁	begin transaction	begin transaction		
t ₂	read (X)	read (Y)	begin transaction	
t ₃	X = X + 50		read (X)	begin transaction
t ₄	write (X)			read (Z)
t ₅		read (X)	X = X - 15	Z = Z + 20
t ₆		X = Y * 12	write (X)	write (Z)
t ₇		write (X)	commit	
t ₈	commit	commit		commit

- Is the schedule in Table 3.1 a serial or non-serial schedule? State your reason.
- Draw the precedence graph for the transactions given in Table 3.1.
- Does the schedule in Table 3.1 have conflict-serializable or not? How can you determine this?
- What problem does the schedule in Table 3.1 face?

i) Non-serial schedule. This is because different transactions are interleaved. A serial schedule would complete one transaction before starting next.



iii) Not conflict-serializable, because it has cycle

iv) The problem faced is lost update. This is because multiple transactions update X concurrently, one transaction's final write can overwrite another.

- b) Consider the schedule transactions in Table 3.2 and answer all following questions (i) – (iv).

Table 3.2: Schedule T11, T12, T13 and T14

Time	T11	T12	T13	T14
t ₁	begin transaction			
t ₂	read (A)	begin transaction		
t ₃		read (B)	begin transaction	
t ₄	A = A + 200	B = B - 50	read (A)	begin transaction
t ₅	write (A)			read (C)
t ₆		write (B)	A = A - 30	C = C + 10
t ₇			write (A)	write (C)
t ₈	commit			
t ₉		read (A)		
t ₁₀		A = A * 2		
t ₁₁		write (A)		
t ₁₂		commit	commit	commit

- Implement the basic locking to the schedule in Table 3.2 by specifying the appropriate lock type (shared or exclusive) for each transaction.
- Based on your answer in (i), state any other issues that may arise.
- Implement the Two-Phase Locking (2PL) protocol to the schedule in Table 3.2.
- Based on your answer in (iii), state any other issues that may arise.

i) T11 - Exclusive lock for A
 T12 - Exclusive lock for B and A
 T13 - Exclusive lock for A
 T14 - Exclusive lock for C

ii) Deadlock, starvation, livelock

iii)

Time	T11	T12	T13	T14
t ₁	begin transaction			
t ₂	write_lock(A)	begin transaction		
t ₃	read(A)	write_lock(B)	begin transaction	
t ₄	A = A + 200	read(B)	write_lock(A)	begin transaction
t ₅	write(A)	B = B - 50	WAIT	write_lock(C)
t ₆		write(B)	WAIT	read(C)
t ₇			WAIT	C = C + 10
t ₈	commit, unlock(A)		WAIT	write(C)
t ₉		write_lock(A)	read(A)	
t ₁₀		WAIT	A = A - 30	
t ₁₁		WAIT	write(A)	
t ₁₂		WAIT	commit, unlock(A)	commit, unlock(C)
t ₁₃		read(A)		
t ₁₄		A = A * 2		
t ₁₅		write(A)		
t ₁₆		commit, unlock(B)		
t ₁₇				
t ₁₈				
t ₁₉				

iv) Deadlock will still arise if transactions wait on each other's locks. other than that is starvation, which transaction requesting an exclusive lock may keep being delayed if others repeatedly acquire locks before it.

QUESTION 4 (15 MARKS)

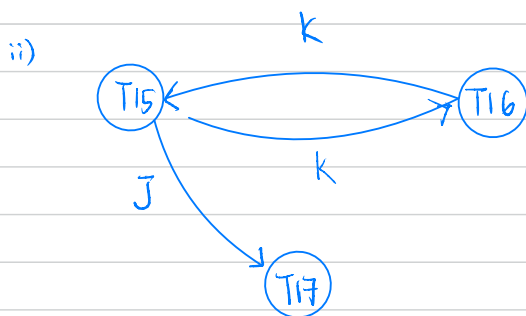
Consider the schedule transactions in Table 4 and answer all following questions (i) – (v).

Table 4.1: Schedule T15, T16 and T17

Time	T15	T16	T17
t_1	begin transaction		
t_2	read (J)	begin transaction	
t_3	$J = J + 10$	read (K)	begin transaction
t_4	write (J)	$K = K - 5$	read (J)
t_5	Read (K)	write (K)	$J = J - 15$
t_6	$K = K + 20$		write (J)
t_7	write (K)	commit	
t_8	commit		commit

- Is the schedule in Table 4 a serial or non-serial schedule? State your reason.
- Draw the precedence graph for the transactions given in Table 4.
- Does the schedule in Table 4 have conflict-serializable or not? How can you determine this?
- What problem does the schedule in Table 4 face?
- Apply the two-phase locking (2PL) protocol to emphasize the positioning of lock and unlock operations in each transaction using the following format.

i) Non-serial schedule because T15, T16 and T17 are interleaved.



iii) Non Conflict-serializable because it has cycle.

iv) Lost update. At t_5 , T15 reads K while T16 writes K at same time. T16 updates K and write it. Later, T15 computes $K = K + 20$ using old value and writes K . This overwrite T16's update, T16's update is lost.

v)

Time	T15	T16	T17
t ₁	begin transaction		
t ₂	write-lock (J)	begin transaction	
t ₃	read (J)	write-lock (K)	begin transaction
t ₄	J = J + 10	read (K)	write-lock (J)
t ₅	write (J)	K = K - 5	WAIT
t ₆	write-lock (K)	write (K)	WAIT
t ₇	WAIT		WAIT
t ₈	WAIT	commit, unlock (K)	WAIT
t ₉	read (K)		WAIT
t ₁₀	K = K + 20		WAIT
t ₁₁	write (K)		WAIT
t ₁₂	commit, unlock (J, K)		WAIT
t ₁₃			read (J)
t ₁₄			J = J - 15
t ₁₅			write (J)
t ₁₆			
t ₁₇			commit-unlock (J)